The bionicreading package

Explorer

2026-04-27

Abstract

bionicreading highlights the leading part of western ASCII words. It provides a small LaTeX3 interface backed by LuaTeX node processing. Both inline commands and paragraph environments mark their contents with a LuaTeX attribute; the Lua backend later converts marked western glyphs during paragraph processing. CJK glyphs and mathematical material are not converted.

1 Overview

The package implements a bionic-reading-style visual treatment for western text. For each ASCII word, a calculated prefix is switched to the bold version of the current font. The calculation follows the fixation tables used by the text-vide project: higher `fixation-point` values generally produce shorter highlighted prefixes.

The package is intentionally scoped to western ASCII words. It does not attempt to segment CJK text, and it does not modify math lists. LuaLaTeX is required because all conversion is performed on the node list after macro expansion.

2 User interface

`\bionicsetup` `\bionicsetup{`⟨*options*⟩`}`

Sets global package options. The currently supported keys are `enabled` and `fixation-point`.

`\bionic` `\bionic[`⟨*options*⟩`]{`⟨*text*⟩`}`

Marks inline or short text for bionic-reading conversion. The command does not scan the argument as a TeX string. It sets a LuaTeX attribute while the argument is typeset; the Lua backend later processes the marked glyph nodes.

\bionicstart `\bionicstart[`⟨*options*⟩`]`
\bionicstop ⟨*text*⟩
`\bionicstop`

Enables and disables marking for an explicit scope. The start command registers the current normal font and its bold counterpart as a font pair, then marks subsequently created glyphs with the current fixation point.

bionicpar `\begin{bionicpar}[`⟨*options*⟩`]`
⟨*paragraphs*⟩
`\end{bionicpar}`

A paragraph environment for bionic reading. The environment uses the same LuaTEX attribute mechanism as `\bionic`, but it also terminates the current paragraph at the end of the environment.

3 Options

Key	Value	Meaning
`enabled`	boolean	Enables or disables conversion.
`fixation-point`	1–5	Selects the fixation table.

4 Fixation-point algorithm

The value of `fixation-point` is not a percentage. It selects one of five boundary tables. During Lua node processing, every marked ASCII word is measured by glyph count. The backend then looks up the first boundary in the selected table that is greater than or equal to the word length.

```
fixation-point = 1: {0, 4, 12, 17, 24, 29, 35, 42, 48}
fixation-point = 2: {0, 2, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29,
                     31, 33, 35, 37, 39, 41, 43, 45, 47, 49}
fixation-point = 3: {0, 2, 4, 5, 6, 8, 9, 11, 14, 15, 17, 18, 20, 0, 21, 23,
                     24, 26, 27, 29, 30, 32, 33, 35, 36, 38, 39, 41, 42, 44,
                     45, 47, 48}
fixation-point = 4: {0, 2, 4, 5, 6, 8, 9, 10, 12, 13, 15, 16, 17, 19, 20, 22,
                     24, 25, 26, 28, 29, 30, 32, 33, 34, 35, 37, 38, 39, 40,
                     42, 43, 44, 45, 47, 48, 49}
fixation-point = 5: {0, 2, 3, 5, 6, 7, 8, 10, 11, 12, 14, 15, 17, 19, 20, 21,
                     23, 24, 25, 26, 28, 29, 30, 32, 33, 34, 35, 37, 38, 39,
                     41, 42, 43, 44, 46, 47, 48}
```

If the matched boundary is at position $\langle i \rangle$, the number of bold glyphs is computed as $\langle \textit{word length} \rangle - (\langle i \rangle - 1)$. The backend then switches the first computed number of glyph nodes to the registered bold font. If the word is longer than all boundaries in the selected table, only the first glyph is bolded.

Higher `fixation-point` values use denser boundary tables. For the same word length, a denser table is usually matched at a later position, so the formula subtracts a larger value and produces a shorter bold prefix.

5 Examples

```
\usepackage{bionicreading}

\bionic{Confucius said: Madam, I'm Adam.}
\bionic[fixation-point=5]{text-vide and internationalization}

\begin{bionicpar}[fixation-point=3]
\lipsum[1]
\end{bionicpar}
```

Both interfaces use the same LuaTEX node backend. Use `\bionic` for inline spans and `bionicpar` for paragraph blocks.

6 LuaTEX backend

The Lua backend scans paragraph node lists in `pre_linebreak_filter`. Only glyphs carrying the package attribute are converted. ASCII letters and digits are accumulated as one word. CJK glyphs, spaces, punctuation, math nodes, and other non-word nodes terminate the current word. Kerning nodes and discretionary hyphenation nodes are not word boundaries: TEX can insert them inside western words, and treating them as boundaries would restart prefix highlighting in the middle of a word.

The backend changes only glyph font IDs. It does not insert new nodes and does not alter the textual content. This keeps macro expansion, CJK text, and math material under TEX's normal control.

7 Implementation

7.1 Package file

```
1 ⟨*package⟩
2 \NeedsTeXFormat{LaTeX2e}
```

```
\ProvidesExplPackage
  {bionicreading}
  {2026-04-27}
  {0.1.0}
  {Bionic-reading style highlighting for ASCII western text}

\RequirePackage{expl3}
\RequirePackage{xparse}

\ExplSyntaxOn

\msg_new:nnn { bionicreading } { invalid-fixation-point }
  {
    Invalid~fixation-point~'#1'.~
    The~value~must~be~an~integer~from~1~to~5.
  }
\msg_new:nnn { bionicreading } { luatex-required }
  { The~bionicreading~package~requires~LuaLaTeX. }

\sys_if_engine_luatex:F
  { \msg_fatal:nn { bionicreading } { luatex-required } }

\RequirePackage{luatexbase}

\bool_new:N \l__bionicreading_enabled_bool
\int_new:N \l__bionicreading_fixation_point_int
\int_new:N \l__bionicreading_normal_font_int
\int_new:N \l__bionicreading_bold_font_int
\newluatexattribute \g__bionicreading_attribute

\keys_define:nn { bionicreading }
  {
    enabled .bool_set:N = \l__bionicreading_enabled_bool,
    enabled .initial:n = true,
    fixation-point .code:n =
      {
        \int_compare:nNnTF {#1} < { 1 }
          { \msg_error:nnn { bionicreading } { invalid-fixation-point } {#1} }
          {
            \int_compare:nNnTF {#1} > { 5 }
              { \msg_error:nnn { bionicreading } { invalid-fixation-point } {#1} }
              { \int_set:Nn \l__bionicreading_fixation_point_int {#1} }
          }
```

```
      },
    fixation-point .initial:n = 1,
  }

\NewDocumentCommand \bionicsetup { m }
  { \keys_set:nn { bionicreading } {#1} }

\cs_new_protected:Npn \__bionicreading_begin:n #1
  {
    \keys_set:nn { bionicreading } {#1}
    \__bionicreading_lua_register_font_pair:
    \__bionicreading_set_attribute:
  }

\NewDocumentCommand \bionic { O{} +m }
  {
    \group_begin:
      \__bionicreading_begin:n {#1}
      #2
    \group_end:
  }

\directlua
  {
    bionicreading = dofile(kpse.find_file("bionicreading.lua"))
    bionicreading.set_attribute(
      luatexbase.attributes["g__bionicreading_attribute"]
    )
    bionicreading.register_callbacks()
  }

\cs_new_protected:Npn \__bionicreading_set_attribute:
  {
    \bool_if:NTF \l__bionicreading_enabled_bool
      {
        \setluatexattribute \g__bionicreading_attribute
          { \int_use:N \l__bionicreading_fixation_point_int }
      }
      { \unsetluatexattribute \g__bionicreading_attribute }
  }

\cs_new_protected:Npn \__bionicreading_lua_register_font_pair:
  {
```

```
    \group_begin:
      \int_set:Nn \l__bionicreading_normal_font_int { \fontid \font }
      \bfseries
      \int_set:Nn \l__bionicreading_bold_font_int { \fontid \font }
      \directlua
        {
          bionicreading.register_font_pair(
            \int_use:N \l__bionicreading_normal_font_int,
            \int_use:N \l__bionicreading_bold_font_int
          )
        }
    \group_end:
  }

\NewDocumentCommand \bionicstart { O{} }
  {
    \group_begin:
      \__bionicreading_begin:n {#1}
  }

\NewDocumentCommand \bionicstop { }
  { \group_end: }

\NewDocumentEnvironment { bionicpar } { O{} }
  { \bionicstart [#1] }
  { \par \bionicstop }

\ExplSyntaxOff
⟨/package⟩
```

7.2 Lua backend

```
⟨*lua⟩
local M = {}

local glyph_id = node.id("glyph")
local math_id = node.id("math")
local disc_id = node.id("disc")
local kern_id = node.id("kern")

local boundaries = {
  { 0, 4, 12, 17, 24, 29, 35, 42, 48 },
  { 1, 2, 7, 10, 13, 14, 19, 22, 25, 28, 31, 34, 37, 40, 43, 46, 49 },
  { 1, 2, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47
```

```
  { 0, 2, 4, 5, 6, 8, 9, 11, 14, 15, 17, 18, 20, 0, 21, 23, 24, 26, 27, 29, 30, 32, 33, 35, 3
  { 0, 2, 3, 5, 6, 7, 8, 10, 11, 12, 14, 15, 17, 19, 20, 21, 23, 24, 25, 26, 28, 29, 30, 32,
}

M.attribute = nil
M.font_map = {}
M.callbacks_registered = false

local function is_ascii_letter(char)
  return (char >= 65 and char <= 90) or (char >= 97 and char <= 122)
end

local function is_ascii_digit(char)
  return char >= 48 and char <= 57
end

local function is_ascii_alnum(char)
  return is_ascii_letter(char) or is_ascii_digit(char)
end

local function fixation_length(word_length, fixation_point)
  local list = boundaries[fixation_point] or boundaries[1]
  for index, boundary in ipairs(list) do
    if word_length <= boundary then
      local length = word_length - (index - 1)
      if length < 0 then
        return 0
      end
      return length
    end
  end
  local length = word_length - #list
  if length < 0 then
    return 0
  end
  return length
end

local function node_marker(n)
  if not M.attribute then
    return nil
  end
  local marker = node.has_attribute(n, M.attribute)
```

```
  if marker and marker >= 1 and marker <= 5 then
    return marker
  end
  return nil
end

local function bold_word(word, has_letter, fixation_point)
  if not has_letter or #word == 0 then
    return
  end
  local bold_count = fixation_length(#word, fixation_point)
  for i = 1, bold_count do
    local glyph = word[i]
    local bold_font = M.font_map[glyph.font]
    if bold_font then
      glyph.font = bold_font
    end
  end
end

function M.process_list(head)
  if not M.attribute then
    return head
  end

  local word = {}
  local has_letter = false
  local fixation_point = nil

  local function flush()
    if fixation_point then
      bold_word(word, has_letter, fixation_point)
    end
    word = {}
    has_letter = false
    fixation_point = nil
  end

  for n in node.traverse(head) do
    local marker = node_marker(n)
    if n.id == glyph_id and marker and is_ascii_alnum(n.char) then
      if fixation_point and fixation_point ~= marker then
        flush()
```

```
      end
      fixation_point = marker
      word[#word + 1] = n
      if is_ascii_letter(n.char) then
        has_letter = true
      end
    elseif n.id == kern_id or n.id == disc_id then
      -- Font kerning and discretionary hyphenation can occur inside a word.
      -- They must not restart fixation calculation for the following glyphs.
    else
      flush()
    end
  end

  flush()
  return head
end

function M.set_attribute(attribute)
  M.attribute = tonumber(attribute)
end

function M.register_font_pair(normal_id, bold_id)
  normal_id = tonumber(normal_id)
  bold_id = tonumber(bold_id)
  if normal_id and bold_id then
    M.font_map[normal_id] = bold_id
  end
end

function M.register_callbacks()
  if M.callbacks_registered then
    return
  end
  if luatexbase and luatexbase.add_to_callback then
    luatexbase.add_to_callback("pre_linebreak_filter", M.process_list, "bionicreading")
  else
    callback.register("pre_linebreak_filter", M.process_list)
  end
  M.callbacks_registered = true
end

return M
```

259 ⟨/lua⟩

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.